

Object Oriented Programming Lab

SPRING 2024

LAB 02



FAST National University of Computer
and Emerging Sciences

Learning Outcomes

In this lab you are expected to learn the following:

- ✓ Pointers
- ✓ Dynamic Memory Allocation

POINTERS

A pointer is a variable that stores an address of a memory location.

Pointer Declaration and initialization

Pointers is declared as follow.

Type* <variable Name>;

```
#include<iostream>
using namespace std;

int main(){

    int *x=NULL;
    // or
    int* x=NULL;
    //or
    int * x=nullptr;

    return 0;
}
```

Address-of operator (&)

The ampersand, **&**, called the **address of operator**, is a unary operator that returns the address of its operand.

```
#include<iostream>
using namespace std;

int main(){

    //declare and initialize string* variable
    string *str=nullptr;
    //declare and initialize string variable
    string temp="hello";
    //store the address of temp to str, both will
    //start pointing to same memory location
    str=&temp;

    //print the address of temp
    cout<<"temp Address "<<&temp<<endl;

    //print value in str
    cout<<"str Value "<<str<<endl;

    return 0;
}
```

```
temp Address 0x7ffdc2d7c440
str Value 0x7ffdc2d7c440
```

Dereference operator (*)

The unary operator * is the dereferencing operator that returns the value of its operand.

```
#include<iostream>
using namespace std;

int main(){

    //declare and initialize string* variable
    string *str=nullptr;
    //declare and initialize string variable
    string temp="hello";
    //store the address of temp to str, both will
    //start pointing to same memory location
    str=&temp;

    //print the address of temp
    cout<<"temp Value "<< temp<<endl;

    //Dereferencing to print value in str
    cout<<"str Value "<< *str<<endl;

    return 0;
}
```

```
temp Value hello
str Value hello
```

Array manipulation using pointer

The identifier of the array is a constant pointer. The elements of the array can be accessed using dereferencing in the following manner.

```
#include<iostream>
using namespace std;

int main(){

    int arr[3]={2,4,8};
    int *p=arr;

    cout<<arr[1]<<endl;
    cout<< *( p+1 )<<endl;

    return 0;
}
```

4
4

Note: You are supposed to pass the test cases for all functions.

Problem 1.

Write a program that asks the user to enter integers as inputs to be stored in the variables 'a' and 'b' respectively. There are also two integer pointers named ptrA and ptrB. Assign the values of 'a' and 'b' to ptrA and ptrB respectively, and display them.

Problem 2.

Given the string "A string." Print on one line the letter on the index 0, the pointer position and the letter t. update the pointer to pointer +2. Then, in another line print the pointer and the letters r and g of the string (using the pointer).

Problem 3.

Create a dynamic memory array and take input from user. The value must be in range of 0-100. Write a function named as calculateGardes which takes an array and its size as function parameters. You are required to apply following grading policy. **Note:** You have to release the memory using delete operator before the end of program.

Function Prototype: char *calculateGrades(int arr[],int size

Score	Grade
91-100	A
76-90	B
60-75	C
51-59	D
Below 50	F

Problem 4:

Write a function named as rotateArray that takes an array of type integer, its size, input n which shifts elements of an array to the right n times, and a number m which divide array into m parts as function parameters. You first need to need to divide array into m number of parts and then shifts elements of an array to the right n times. Array under consideration is circular.**Function Prototype:** rotateArray (int *arr, int sizeofArray, int n, int m)

Example:

Input:

1	2	3	4	5	6
---	---	---	---	---	---

Output: Input n=2, m=2

2	3	1	5	6	4
---	---	---	---	---	---

Problem 5:

You have a program that uses a dynamic array to store integers. The user is allowed to input integers until they enter a specific sentinel value, say -1. Implement a scenario where the dynamic array resizes itself when it reaches its current capacity. Ensure the program doesn't lose any data during the resizing process.

Submission Instructions:

1. Create a single cpp file containing all the functions of the problems and main function. Save the cpp file as Q1.cpp.
2. Now create a new folder with name ROLLNO_SEC_LAB01 e.g. i22XXXX_A_LAB01.
3. You need to display your roll no and name before the output of each question.
4. comment out the main Function and then move to this newly created directory and compress it into a .zip file.
5. Now you have to submit this zipped file on Google Classroom.

6. If you don't follow the above-mentioned submission instruction, you will be marked zero.
7. Plagiarism in the Lab Task will result in zero marks in the whole category.